

■ AI CHAT BOT BASED BUS TICKETING SYSTEM

Project Abstract & Technical Description

Version 1.0 • June 2026 • Confidential

3+ <i>AI Models</i>	10 <i>Bus Scrapers</i>	15+ <i>Indian Cities</i>	4 <i>Smart Filters</i>
-------------------------------	----------------------------------	------------------------------------	----------------------------------

An intelligent, real-time bus ticket discovery platform powered by AI-driven natural language understanding, live web scraping from RedBus, and a premium glassmorphism React interface — enabling travellers to find, compare and book buses across South India through simple conversational queries.

ABSTRACT

The **AI Chat Bot Based Bus Ticketing System** is a full-stack web application that reimagines how travellers discover and book bus tickets in India. Instead of navigating complex booking portals, users interact with a smart conversational AI assistant that interprets natural-language queries such as *"Find me the cheapest AC bus from Madurai to Chennai tomorrow"* and instantly returns filtered, sorted, real-time bus data sourced directly from RedBus.

The system integrates a **multi-model AI pipeline** using OpenRouter (supporting models such as Nous Hermes 3 Llama-3.1-405B) to extract travel intent — source city, destination city, travel date, bus-type preference, and sorting criteria — from unstructured user messages. A robust **rule-based fallback parser** ensures 100% reliability even when AI rate-limits are encountered.

On the data side, a Playwright-powered scraper retrieves live bus listings, seat availability, pricing, operator details, and booking URLs from RedBus's internal GraphQL API. The backend, built with **FastAPI (Python)**, applies intelligent filter-and-sort logic before returning results, ensuring queries like "sleeper bus" or "cheapest" surface truly relevant options at the top.

The frontend is a **React + TypeScript (Vite)** single-page application featuring a 2026-style **glassmorphism UI** with a dark/light theme toggle (persisted via localStorage), animated badge highlights for active filters (*■ AC, ■■ Sleeper, ■ Cheapest), a smart connecting-route suggester for indirect journeys, an AI-powered Travel Guide panel, an interactive map, and a complete booking history tracker.

Together, these components deliver a seamless, end-to-end travel search experience that reduces the cognitive load of ticket booking to a single natural-language prompt.

Keywords: *Conversational AI, Bus Ticketing, Natural Language Processing, Web Scraping, FastAPI, React, Playwright, OpenRouter, Glassmorphism UI, Real-time Data, Smart Filters*

1. INTRODUCTION

Online bus ticket booking in India is fragmented across multiple platforms (RedBus, AbhiBus, MakeMyTrip, etc.), each with its own search interface and filtering mechanisms. First-time travellers and non-technical users often struggle to compare options across operators, seat types, and prices.

This project addresses that friction by providing a **unified AI chat interface** that acts as a personal travel assistant. Users describe their journey in plain English; the system does all the heavy lifting — parsing intent, fetching live data, applying smart filters, and presenting results in a visually rich dashboard.

2. SYSTEM ARCHITECTURE

The system follows a classic **three-tier architecture**: a React frontend, a FastAPI backend, and external data sources accessed via scraping.

Component	Technology / Library
Frontend (Client)	React 18 + TypeScript + Vite + Vanilla CSS (Glassmorphism)
Backend (Server)	Python 3.11+ · FastAPI · Uvicorn ASGI
AI Engine	OpenRouter API (Nous Hermes 3 / Llama 3.1 405B) + Rule-based fallback
Web Scraper	Playwright (async) · RedBus GraphQL API interception
State / Storage	React useState / useEffect · localStorage (theme + history)
Deployment	Docker-ready · CORS-enabled · .env configuration

3. CORE FEATURES

- **Conversational AI Search** — Users type or speak natural-language queries. The AI extracts source, destination, date, filter, and sort preference in a single call or through a guided multi-turn dialogue.
- **Smart AI Filtering** — Detects keywords such as 'cheapest', 'AC bus', 'sleeper', 'Volvo', 'fastest', 'night bus' and applies them server-side before returning results, so the top card always matches the user's preference.
- **Live Bus Data** — Playwright scrapes RedBus's live JSON API on every fresh query (10-second cache for repeated queries), returning real prices, seat availability, operator names, departure/arrival times, and booking links.
- **Smart Connecting Routes** — If no direct bus exists, the system automatically suggests a two-leg route via a known intermediate city (e.g., Chennai → Dindigul → Kodaikanal).
- **Premium Glassmorphism UI** — 2026-style dark/light theme with CSS variables, backdrop-filter blur panels, animated filter badges, micro-animations, and a persistent theme toggle stored in localStorage.
- **Travel Guide Panel** — AI-generated travel tips, famous places, food recommendations, and best time to visit for the destination city.
- **Interactive Map** — Leaflet.js map highlighting tourist spots near the destination, sourced via curated data.
- **Booking History & Reviews** — In-browser booking tracker with confirmation IDs, and an operator review system with star ratings.
- **Voice Input** — Web Speech API integration for hands-free search.
- **User Personalisation** — Greeting, frequent-route suggestions, and AI-powered quick-reply chips based on past searches.

4. AI INTENT PIPELINE

The AI pipeline is designed for **maximum reliability under free-tier rate limits**:

- **Step 1:** User message arrives at the `/search` FastAPI endpoint along with conversation history (last 6 messages) and partial context (e.g., already-known city).
- **Step 2:** A **system prompt** instructs the AI to return a strict JSON object with fields: `from_city`, `to_city`, `date`, `passengers`, `filter`, `sort_by`.
- **Step 3:** The primary model (Nous Hermes 3 Llama-3.1-405B via OpenRouter) is called. On rate-limit (HTTP 429) or empty response, the system automatically retries the next available model from the fallback chain.
- **Step 4:** If all AI models fail, the **rule-based parser** applies regex patterns to extract city names, dates (relative: today, tomorrow, weekend), filter keywords (ac, sleeper, cheapest, volvo, night), and sort preferences.
- **Step 5:** The parsed **collected** object is checked for completeness. Missing fields trigger an `ask_details` response with quick-reply chips (Tomorrow / Day After / This Weekend).
- **Step 6:** Once all required fields are present, the system calls the scraper, applies `filter_and_sort_buses()`, and returns the enriched ticket list.

5. FULL TECHNOLOGY STACK

Component	Technology / Library
Language (Backend)	Python 3.11+
Web Framework	FastAPI 0.110+ · Pydantic · Uvicorn
AI / LLM	OpenRouter API · Nous Hermes 3 Llama 3.1 405B (free tier)
Web Scraping	Playwright (async) · httpx · BeautifulSoup
Language (Frontend)	TypeScript 5 · React 18
Build Tool	Vite 6 · ESBuild
Styling	Vanilla CSS · CSS Custom Properties · Glassmorphism
Mapping	Leaflet.js · React-Leaflet
Icons	Lucide React
Voice	Web Speech API
HTTP Client	Fetch API · httpx (async)
Persistence	localStorage (theme, history, sessions)
Containerisation	Docker · Docker Compose
Version Control	Git

6. DATA FLOW

1. **User Input** → React UI sends POST /search with {query, context, history}.
2. **Intent Parsing** → FastAPI calls AI/rule-based parser → returns collected JSON.
3. **Data Fetch** → Playwright intercepts RedBus GraphQL API → returns bus list.
4. **Filter & Sort** → filter_and_sort_buses() applies ac/sleeper/cheapest logic.
5. **Response** → Returns {type, data[], intent, active_filter, sort_by, search_summary}.
6. **UI Update** → React sets activeFilter state → badge & card highlights rendered.
7. **Side Effects** → Travel Guide, booking history, chat transcript all updated.

7. LIMITATIONS & FUTURE SCOPE

Current Limitations:

- Free-tier AI models (OpenRouter) are subject to rate limits; the rule-based parser provides a reliable fallback but has reduced semantic understanding.
- Web scraping is dependent on RedBus DOM/API structure; changes to the target site may require scraper updates.
- No real payment gateway integration; booking redirects to the third-party site.
- Bus data is limited to routes supported by RedBus India.

Future Enhancements:

- Integrate a fine-tuned domain-specific NLP model for richer intent extraction.
- Add support for train, flight, and cab bookings via additional scrapers.
- Real-time seat-map visualisation and live seat-selection within the app.
- Payment gateway integration (Razorpay / Stripe) for end-to-end booking.
- Progressive Web App (PWA) with offline caching and push notifications.
- Multi-language support (Tamil, Hindi, Kannada) for regional user accessibility.
- Backend database (PostgreSQL) for persistent user profiles and booking records.
- Price prediction using historical data and ML regression models.

8. CONCLUSION

The AI Chat Bot Based Bus Ticketing System successfully demonstrates how conversational AI, real-time web scraping, and modern frontend engineering can be combined to simplify a complex real-world problem. The system transforms a traditionally friction-heavy bus booking experience into a natural, zero-learning-curve interaction available at <http://localhost:5173>.

By using a layered AI approach — primary LLM → model fallback → rule-based parser — the system achieves near-100% uptime for intent extraction while remaining cost-free on the AI inference side. The premium glassmorphism UI, smart filter badges, connecting-route intelligence, and Travel Guide panel elevate it well beyond a simple search tool into a comprehensive travel companion.

This project serves as a strong foundation for a production-grade, AI-first travel platform targeting India's rapidly growing online ticketing market.

Generated on 05 June 2026, 17:10 • Generated on 05 June 2026, 17:10 | AI Chat Bot Based Bus Ticketing System | For academic / presentation use